

Diligent Code Studio Operator/User Manual

Community Edition

v0.7.0-dev

Secure software-building workbench for local-first development workflows

Updated: June 1, 2026

Document Control

Field	Value
Product	Diligent Code Studio Community Edition
Manual version	v0.7.0-dev
Audience	Operators, end users, testers, maintainers, and release builders
Scope	Installation, setup, editor usage, AI assistance, GitHub quality builds, troubleshooting, and release artifacts
Recommended status	Development milestone / beta candidate, not final production release

Table of Contents

1. Overview
2. What is new in v0.7.0-dev
3. System requirements
4. Installation and launch
5. First Run Setup Wizard
6. Setup and dependency checks
7. Workspace management
8. Editor page
9. AI Help and provider setup
10. Terminal, Git, diagnostics, and logs
11. Manual and help system
12. GitHub Quality Build and artifacts
13. Operator procedures
14. Troubleshooting
15. Security and privacy notes
16. Release checklist
17. Appendix A - Useful commands
18. Appendix B - File locations

Note

This is a static table of contents for reliable PDF export. Use the section headings to navigate in Word or PDF readers.

1. Overview

Diligent Code Studio is a local-first AI development workbench designed to help users edit code, manage a workspace, run diagnostics, configure development dependencies, use AI assistance, and produce release packages from one desktop application.

- Local-first design: the app is intended to run on the user workstation and operate against local project folders.
- Editor-centered workflow: the Editor page is the primary work area for viewing and editing project files.
- Guided setup: the First Run Setup Wizard helps users configure dependencies, workspace path, manual access, and AI settings.
- Quality-focused release process: GitHub Actions Quality Build validates and packages Windows, Linux, and macOS artifacts.

v0.7.0-dev positioning

This manual describes the v0.7.0-dev milestone. Treat this build as a working development/beta checkpoint until downloaded artifacts have been tested on clean machines.

2. What is new in v0.7.0-dev

Area	Update	Benefit
First Run Setup	Setup wizard actions were expanded.	Users can jump to setup, AI settings, manual, and workspace tasks more directly.
Dependency checks	Dependency checks can run automatically after startup.	Less user interaction is required to know whether the workstation is ready.
Workspace startup	Saved workspace paths auto-open after preferences load.	The file tree and terminal directory become active without reselecting the folder.
Editor layout	The Editor page opens with a maximized coding area and a compact footer/status strip.	More usable code space and less wasted bottom area.
Manual archive	v0.7.0 manual copies are archived under public/manuals and docs.	Quality checks and source users can find the matching manual version.
GitHub Actions	Quality Build supports branch triggers and artifact staging for Windows, Linux, and macOS.	Build outputs are easier to verify and download.

3. System requirements

3.1 User workstation

- Windows 10/11 is the primary development and testing environment.
- Linux and macOS builds are produced through GitHub Actions artifacts and should be tested on target systems.
- A writable workspace folder, for example C:\DiligentProjects.
- Internet access for dependency installation, GitHub repository operations, and optional AI provider connectivity.

3.2 Development dependencies

- Node.js and npm for frontend build and scripts.
- Rust toolchain and Cargo for Tauri backend builds.
- Microsoft Visual Studio C++ Build Tools on Windows for native build support.
- Git and GitHub CLI for repository and workflow operations.
- Optional: PowerShell 7, Ollama, and AI provider tooling depending on the selected workflow.

Automatic checks

v0.7.0-dev can run dependency checks automatically after startup and update the Setup health/status area without requiring the user to open Setup first.

4. Installation and launch

4.1 Install from downloaded artifact

1. Download the correct artifact from the GitHub Actions Quality Build summary.
2. Extract the artifact if GitHub downloaded it as a zip archive.
3. Run the installer or package for your operating system.
4. Launch Diligent Code Studio.
5. Complete or review First Run Setup when prompted.

4.2 Run from source during development

```
cd C:\DiligentProjects\DiligentCodeStudio_GitHub
```

```
npm install
```

```
npm run build
```

```
npm run tauri:dev
```

Preferred validation command

Before pushing or packaging, use `npm run quality` to run typecheck, smoke tests, unit tests, security tests, frontend build, and npm audit checks.

5. First Run Setup Wizard

The First Run Setup Wizard is designed to help a new user get from first launch to a usable workspace with minimal guessing.

- Open Setup Page: jumps to setup and dependency tools.
- Check Now: runs the dependency checker from the wizard.
- Choose Workspace Folder: selects a project/workspace folder.
- Open Default Path: attempts to open the configured default workspace path.
- Open AI Settings: jumps to AI configuration settings.
- Open Manual: opens the embedded Operator/User Manual PDF.
- Finish and Open Start Here: completes onboarding and opens the Start Here dashboard.

Resetting the wizard

The wizard can be reset from Settings if an operator wants to repeat first-run onboarding. Resetting the wizard does not erase project files or API keys.

6. Setup and dependency checks

6.1 Automatic startup dependency check

v0.7.0-dev can start a dependency check automatically shortly after launch. The check runs in the background and updates the Setup health strip and Setup page results.

- The user does not need to open the Setup page first.
- Manual Check Again remains available for refreshes.
- Installer buttons should ask for confirmation before starting an install action.
- The Activity Log records dependency check status and failures.

6.2 Recommended operator checks

6. Launch the app and wait for the startup check to complete.
7. Open Setup and confirm dependency results are populated.
8. Use Check Again after installing or updating tools.
9. Confirm missing dependency items provide a clear install or guidance action.

7. Workspace management

7.1 Saved workspace auto-open

v0.7.0-dev improves workspace startup behavior. A saved default or last workspace path is now watched after preferences load, then opened automatically.

- The saved path becomes the active workspace without requiring the user to reselect the folder.
- The file tree loads automatically.
- The terminal working directory is synchronized to the workspace path.
- The Activity Log records messages such as Saved workspace watcher opened.

Expected behavior

If the workspace path is visible in the settings/dialog field, the app should also activate it after startup and populate the file tree.

7.2 Selecting or changing workspace folder

10. Open the workspace folder selector.

11. Choose the desired project folder.
12. Confirm the file tree updates immediately.
13. Close and reopen the app to verify the saved workspace auto-opens.

8. Editor page

8.1 Editor layout

The Editor page is intended to be the main daily work area. The coding area should open maximized by default, with the footer/status strip kept compact.

- The open file tab appears above the editor canvas.
- The Monaco editor/code surface fills the main available page area.
- The footer/status strip stays one compact line for file/path/hash/status information.
- The editor should show many lines of code, not a single collapsed line.

If code disappears

If a future CSS change causes tabs to show but the code area to disappear, verify that editor-page-content, editor-wrap, editor-ai-layout, and editor-canvas all have a definite height. Monaco is rendered with height=100%, so its parent chain must not collapse.

8.2 Recommended editor test

14. Open a real source file from the workspace tree.
15. Confirm the file tab is visible.
16. Confirm code text is visible in the main editor area.
17. Confirm the footer/status area does not consume the bottom half of the page.
18. Resize the application window and confirm the editor remains usable.

9. AI Help and provider setup

AI Help is designed to support coding, explanation, deployment planning, troubleshooting, and guided navigation. It should remain optional and configurable.

- Provider options may include disabled mode, local provider options, or configured external AI provider support.
- The AI Help panel can open, close, and remember placement depending on app settings.
- Context selection should be intentional: current file, selection, project, problems, terminal, or Git context.
- API keys and secrets should not be stored insecurely in saved preferences.

Security

Do not paste API keys, passwords, private certificates, or production secrets into prompts. Keep AI ignore paths current so generated folders, secrets, node_modules, target, dist, and .git content are excluded from context.

10. Terminal, Git, diagnostics, and logs

10.1 Terminal

- The terminal working directory should sync to the active workspace.
- Command execution should validate inputs and avoid unsafe defaults.
- Terminal output should remain scrollable and clear enough for troubleshooting.

10.2 Git page

- Use Git status before commit/push operations.
- Review staged files before committing.
- Avoid committing temporary backup files, diagnostics, or build cache folders.
- Use clear commit messages tied to the feature or fix.

10.3 Activity Log

- Startup dependency checks should log start/success/failure.
- Saved workspace auto-open should log watcher activity.
- AI Help and setup actions should log useful messages for operators.

11. Manual and help system

The manual is bundled as a PDF under public/manuals so the app can open it from the Manual button and Help views. Versioned copies are also archived for release and test coverage.

Location	Purpose
public/manuals/ DiligentCodeStudio_UserManual.pdf	Current manual opened by the app.
public/manuals/ DiligentCodeStudio_UserManual_v0.7.0.pdf	Versioned app-bundled manual archive.
public/manuals/ DiligentCodeStudio_UserManual_v0.7.0_Operator_User_Manual.pdf	Versioned operator/user manual archive.
docs/DiligentCodeStudio_UserManual_v0.7.0.pdf	Source-user manual archive expected by unit tests.
docs/manuals/ DiligentCodeStudio_UserManual_v0.7.0.pdf	Organized documentation archive copy.

Quality requirement

The smoke and unit tests expect matching versioned/cumulative manual archives for the active product version.

12. GitHub Quality Build and artifacts

12.1 Purpose

The Quality Build workflow validates the project and produces downloadable artifacts for supported operating systems.

- Runs TypeScript checks, smoke tests, unit tests, security tests, frontend build, and npm audit.
- Runs Rust compile checks and Rust unit tests.
- Builds Tauri desktop packages.
- Stages build artifacts into dist-artifacts/<platform>.
- Uploads Windows, Linux, and macOS artifact bundles with checksums.

12.2 Expected artifacts

Artifact	Expected platform
diligent-code-studio-0.7.0-dev-windows	Windows package output
diligent-code-studio-0.7.0-dev-linux	Linux package output
diligent-code-studio-0.7.0-dev-macos	macOS package output

12.3 macOS runner note

The macOS job should use a supported current GitHub-hosted macOS runner label. For Intel macOS packaging, macos-15-intel is preferred over older retired labels such as macos-13.

13. Operator procedures

13.1 Daily startup procedure

19. Launch Diligent Code Studio.
20. Allow the automatic dependency check to complete.
21. Confirm the saved workspace opens automatically.
22. Open the Editor page and confirm code is visible.
23. Review any Activity Log warnings before continuing work.

13.2 Pre-push procedure

```
git status --short
npm run quality
npm run tauri:build
git add <changed files>
git commit -m "Describe the change"
git push origin feature/v0.7.0-first-run-wizard
```


14. Troubleshooting

Symptom	Likely cause	Recommended action
Workspace path is visible but file tree is empty	Saved path was not activated	Confirm v0.7.0 saved workspace watcher is present and check Activity Log for watcher messages.
Editor tab is visible but code is missing	Monaco parent container collapsed	Check editor-page-content, editor-wrap, editor-ai-layout, and editor-canvas height rules.
Footer takes half the Editor page	Footer/status strip CSS expanded or wrong container targeted	Use targeted editor layout rules, not global app-shell rules.
Quality Build has no artifacts	Packaging/upload step skipped or path mismatch	Stage artifacts into dist-artifacts/<platform> and set upload-artifact if-no-files-found: error.
macOS job waits forever	Unsupported or unavailable runner label	Use a current supported macOS runner label such as macos-15-intel.
Smoke/unit tests fail for manual	Missing versioned manual archive	Copy current manual into public/manuals and docs archives for the active version.
Mojibake/corrupted characters appear	Bad encoding or copied rich text symbols	Replace with ASCII-safe text and save UTF-8 without BOM.

15. Security and privacy notes

- Do not persist OpenAI or other provider API keys in plain saved preferences.
- Keep AI ignore lists current for .env, node_modules, target, dist, .git, build output, certificates, and secret files.
- Validate terminal command inputs before backend execution.
- Do not commit backup, diagnostic, or generated cache files.
- Run npm audit through npm run quality before release packaging.
- Review Git diffs before pushing workflow or release changes.

16. Release checklist

Check	Status guidance
Version sync	package.json, package-lock.json, Cargo.toml, and tauri.conf.json use numeric package

	version 0.7.0. UI may display 0.7.0-dev.
Manual archives	Current and versioned PDF manuals exist under public/manuals and docs.
Quality build	Windows, Linux, and macOS matrix jobs complete successfully.
Artifacts	All three platform artifacts are downloadable and contain checksum files.
Windows install test	Install or launch downloaded Windows artifact on a clean or separate machine if possible.
Workspace startup	Saved workspace auto-opens and terminal path syncs.
Editor layout	Editor code area is visible, maximized, and footer is compact.
Mojibake check	No corrupted characters appear in header, wizard, AI Help, or manual text.

17. Appendix A - Useful commands

npm run quality

npm run build

npm run tauri:build

npm run typecheck

npm run test:smoke

npm run test:unit

npm run test:security

npm run check:rust

npm run test:rust

gh workflow run quality-build.yml --ref feature/v0.7.0-first-run-wizard

gh run list --workflow quality-build.yml --branch feature/v0.7.0-first-run-wizard --limit 5

18. Appendix B - File locations

Path	Description
src/App.tsx	Main React application logic. Future work should split this into smaller components.
src/styles.css	Application styling and layout rules.
public/manuals	Manual PDF files bundled with the app.

docs and docs/manuals	Source-user manual archives.
scripts/smoke-test.mjs	Smoke test expectations.
scripts/unit-test.mjs	Unit tests, including manual archive expectations.
scripts/security-check.mjs	Security checks for local-first desktop risks.
.github/workflows/quality-build.yml	Quality Build matrix workflow and artifact upload process.
src-tauri	Tauri backend and packaging configuration.

Operator recommendation

For v0.7.0-dev, do not mark the release final until the downloaded Windows, Linux, and macOS artifacts have been tested. This build is a strong development milestone and beta candidate.